Exercice pratique : écrire un test qui protège le code

Le but : installer Vitest + React Testing Library, écrire 2 tests simples sur un composant, les voir passer au **vert**, puis casser le code pour voir un test passer au **rouge**.

Étape 1 — Installer les outils de test

> **Aide :** dans le terminal, à la racine du projet :
>
> `npm install --save-dev vitest @testing-library/react @testing-library/jest-dom jsdom`
>
> (Vitest = le lanceur de tests ; Testing Library = pour manipuler les composants React ; jsdom = un faux navigateur dans lequel les tests s'exécutent.)

Étape 2 — Configurer l'environnement de test

Il faut dire à Vitest de simuler un navigateur.

> **Aide :** dans le fichier `vite.config.js`, ajoutez un bloc `test` :
>
> ```
> import { defineConfig } from 'vite'
> import react from '@vitejs/plugin-react'
>
> export default defineConfig({
>   plugins: [react()],
>   test: {
>     environment: 'jsdom',
>     globals: true,
>     setupFiles: './src/setupTests.js',
>   },
> })
> ```
>
> Puis créez le fichier `src/setupTests.js` contenant une seule ligne :

```
import '@testing-library/jest-dom'
```

Étape 3 — Ajouter le script de test

> **Aide :** dans le `package.json`, section `"scripts"`, ajoutez :
> `"test": "vitest"`
>
> (En le lançant, Vitest reste actif et relance les tests automatiquement à chaque enregistrement — pratique pour voir le vert/rouge en direct.)

Étape 4 — Écrire vos deux premiers tests

Créez un fichier `src/App.test.jsx` à côté de `App.jsx`.

Étape 5 — Lancer les tests et voir le vert

Étape 6 — Casser le code pour voir le rouge (le moment important)

Simulez un bug dans le **composant** (pas dans le test) et observez le filet de sécurité se déclencher.

> Puis **remettez le code correct** : le test repasse au vert.